

What Happens Before Decoding? Prefill Determines GUI Grounding in VLMs

Jiaping Lin¹ Fei Shen² Junzhe Li³ Ping Nie⁴ Fei Yu¹ Ming Li^{1*} Haizhou Li⁵

¹Guangming Laboratory ²National University of Singapore

³Peking University ⁴University of Waterloo

⁵The Chinese University of Hong Kong (Shenzhen)

Abstract

Existing training-free approaches for GUI grounding often rely on multiple inference runs, such as iterative cropping or candidate aggregation, to identify target elements. Despite this additional computation, each forward pass still independently interprets the instruction and parses the visual layout, without enabling progressive interaction among visual tokens. In this paper, we study what happens during GUI grounding in Vision-Language Models (VLMs) and identify a previously overlooked bottleneck. We show that grounding follows a two-stage paradigm: the *prefill stage* determines candidate UI elements, while the *decoding stage* subsequently refines the final coordinates. This asymmetry establishes prefill as the critical step, as errors in candidate selection cannot be effectively corrected during decoding. Based on this observation, we propose **Re-Prefill**, a training-free method that revisits inference by introducing an attention-guided second prefill stage to refine target selection. Specifically, visual tokens that consistently receive high attention from the query position, *i.e.*, the final token, across layers are extracted as a preliminary target hypothesis and appended to the input, together with the instruction hidden states, enabling the model to deeply re-think its decision before coordinate generation. Experiments across four VLMs and five benchmarks, including ScreenSpot-Pro, ScreenSpot-V2, OSWorld-G, UI-Vision, and MMBench-GUI, demonstrate consistent improvements without additional training, with gains of up to 4.3% on ScreenSpot-Pro. Code will be available at <https://github.com/linjiaping1/Re-Prefill>.

1 Introduction

GUI grounding aims to predict the coordinates of a target UI element from a screenshot guided by a natural language instruction, and is a fundamental capability for autonomous agents [1–3]. Vision-Language Models (VLMs) have become the dominant approach to this task, achieving strong performance through large-scale supervised and reinforcement training [4–7]. However, such performance relies on extensive annotation and substantial computing, leading to high labor and computational costs [8–10].

To reduce these costs, recent work has explored training-free methods that enhance grounding at inference time [11, 12]. These approaches typically rely on multiple inference runs, such as iterative cropping to refine visual resolution [12–14] or aggregating predictions across runs [15–17]. For example, ZoomClick [13] characterizes key properties of zooming and performs iterative crop-and-zoom to increase the effective resolution around targets. CoG [14] uses context to refine iterative cropping, thereby improving grounding precision. MVP [16] generates predictions from multiple

*Corresponding author.

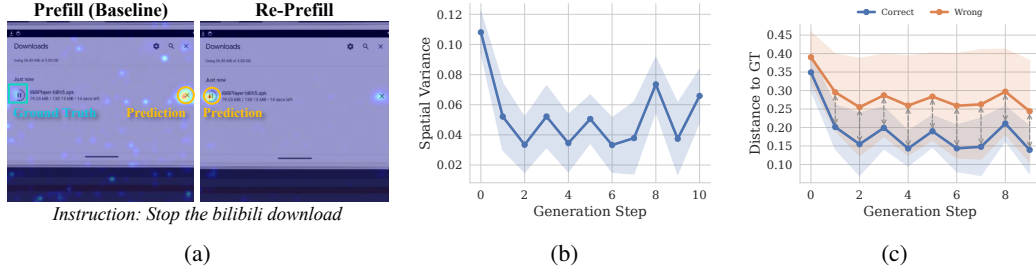


Figure 1: **Prefill vs. Re-Prefill vs. Decoding.** (a) Query-position attention heatmaps over visual tokens. Re-Prefill produces a sharper, more focused distribution that disambiguates the correct target from other candidates. Additional visualizations are provided in Appendix C. (b) Spatial variance of query-position attention across generation steps. The sharp drop after the first generated token shows that target selection is largely completed during the prefill stage. (c) Deviation of the attention centroid from the ground-truth, separated into correct (blue) and incorrect (orange) predictions. The persistent gap in incorrect cases indicates that prefill errors cannot be corrected during decoding. Extended analysis across multiple models and benchmarks is provided in Appendix D.

cropped views and aggregates them to achieve more stable grounding accuracy. While effective, they incur substantial computational overhead. More importantly, our analysis reveals that stacking multiple runs does not enable interaction between visual tokens across runs, and therefore cannot correct errors introduced in earlier passes.

In this work, we attempt to address a more fundamental question: *where do grounding errors originate within inference?* We investigate this question by asking: *what happens before decoding?* We perform a thorough experimental analysis by tracing the query-position attention, *i.e.* the attention from the final token to the visual tokens, at each generation step. We then compute both its spatial variance, reflecting the concentration of attention over image regions, and the deviation of its centroid from the ground-truth target. In Figure. 1, we show the visual-token attention distribution from the query position at the prefill stage and the spatial variance of attention, the deviation of the attention centroid from the ground truth *w.r.t.* the decoding step. Step $t = 0$ on the horizontal axis corresponds to the prefill stage, while $t = 1$ indicates the start of decoding. The results show that during prefill, attention is scattered on a couple of candidate regions, coarsely determining the target element scope. During decoding, coordinates are generated autoregressively by concentrating on one specific region, which can be demonstrated by attention variance decrease. Importantly, the deviation from the ground truth remains large for incorrect predictions, indicating that errors introduced during prefill persist throughout decoding.

These observations establish the prefill stage as the decisive step for grounding, where accurate target selection is critical for subsequent decoding. Based on this insight, we propose **Re-Prefill**, a training-free method that revisits inference by introducing an attention-guided second prefill stage to refine attention concentration and improve target selection. Specifically, after the initial prefill computation, we identify visual tokens that consistently receive high attention from the query position across layers, which represent the model’s preliminary target hypothesis. These tokens, as well as the encoded instruction hidden states, are appended to the original input for re-prefill computation, enabling the model to re-evaluate its target selection before coordinate generation. The illustration in Figure 1 illustrates that Re-Prefill improves target selection by enabling the model to focus on the correct UI element before decoding.

We evaluate Re-Prefill on four VLMs of varying scales [6, 7, 18] across five GUI grounding benchmarks, including ScreenSpot-Pro [19], ScreenSpot-V2 [9], OSWorld-G [20], UI-Vision [21], and MMBench-GUI [22]. Re-Prefill consistently improves grounding accuracy across all models and benchmarks without additional training, achieving gains of up to 4.3% on ScreenSpot-Pro and 2.7% on OSWorld-G.

Our contributions in this work are summarized as follows:

- We identify a two-stage inference paradigm in GUI grounding, where the prefill stage determines target selection and the decoding stage generates coordinates conditioned on it.

- We propose Re-Prefill, a training-free method that improves target selection by revisiting inference with an attention-guided second prefill stage, orthogonal to existing GUI grounding methods.
- We demonstrate consistent improvements across models and benchmarks, establishing prefill as a key bottleneck and providing a new perspective for studying GUI grounding.

2 Revisiting GUI Grounding Inference

2.1 Preliminaries

GUI grounding aims to predict the coordinates (x, y) of a target UI element given a screenshot I and a natural language instruction q . A decoder-only VLM processes the input as a sequence of tokens composed of three segments: system prompt tokens $\mathbf{S}$, visual tokens $\mathbf{V}$, and instruction tokens $\mathbf{T}$. The concatenated input is denoted as $\mathbf{x} = [\mathbf{S}; \mathbf{V}; \mathbf{T}]$.

Inference consists of two stages. In the *prefill stage*, the full sequence $\mathbf{x}$ is processed through L decoder layers in a single forward pass, where self-attention is applied over all tokens and the resulting key-value pairs are stored in a KV cache $\mathcal{C}$. We denote this process as $f_{\text{prefill}}(\mathbf{x}) \rightarrow (\mathbf{h}_q, \mathcal{C})$, where $\mathbf{h}_q$ is the hidden state at the final token position (the query position), which is used to initiate coordinate prediction. In the subsequent *decoding stage*, coordinate tokens are generated autoregressively, with each step conditioned on the KV cache and previously generated tokens: $y_t = f_{\text{decode}}(y_{<t}, \mathcal{C})$.

2.2 Prefill as the Bottleneck of GUI Grounding

To better understand where grounding errors originate, we analyze the attention dynamics of VLMs during inference. In particular, we investigate two key questions: (1) when target region selection is determined, and (2) whether an incorrect selection can be corrected during decoding.

We employ a unified probing mechanism based on query-position attention. Let $\mathbf{a}_t \in \mathbb{R}^{N_v}$ denote the attention from the query position to visual tokens at generation step t , where $t = 0$ corresponds to the prefill stage and $t \geq 1$ to decoding steps. To reduce layer-wise variation, $\mathbf{a}_t$ is averaged across all decoder layers. All results are reported on ScreenSpot-Pro with Qwen3-VL-8B-Instruct, with extended analysis across multiple models and benchmarks provided in Appendix D.

Early attention concentration determines target selection. We first examine how attention evolves spatially across generation steps. The spatial concentration of $\mathbf{a}_t$ is measured by its variance,

$$\sigma_t^2 = \sum_{i=1}^{N_v} a_t(i) \|\mathbf{p}_i - \boldsymbol{\mu}_t\|^2, \quad \boldsymbol{\mu}_t = \sum_{i=1}^{N_v} a_t(i) \mathbf{p}_i, \quad (1)$$

where $\mathbf{p}_i$ is the 2-D coordinate of v_i and $\boldsymbol{\mu}_t$ is the attention centroid. A large σ_t^2 indicates dispersed attention, while a small value indicates concentration on a localized region. As shown in Figure 1(b), σ_t^2 is high at $t = 0$, suggesting that attention during prefill is distributed over multiple candidate regions. At $t = 1$, the variance drops sharply, indicating that attention rapidly concentrates on a single region once decoding begins. Thereafter, the variance remains low with only minor fluctuations.

This behavior suggests that target region selection is largely completed during the prefill stage, and that decoding primarily refines coordinates within the selected region. The small fluctuations during decoding likely arise from attending to local contextual layout, which helps improve coordinate precision. Similar patterns are observed across multiple benchmarks, indicating that this behavior is consistent across models and datasets.

Prefill errors persist throughout decoding. We next examine whether incorrect target selection can be corrected during decoding. Samples are divided into Correct and Wrong groups according to whether the predicted coordinates fall inside the ground-truth bounding box. For each group, we measure the normalized distance between the attention centroid $\boldsymbol{\mu}_t$ and the ground-truth center $\mathbf{g}$:

$$d_t = \frac{\|\boldsymbol{\mu}_t - \mathbf{g}\|}{D}, \quad (2)$$

where D is the image diagonal. As shown in Figure 1(c), both groups exhibit a decrease in distance from $t = 0$ to $t = 1$, consistent with the attention concentration behavior observed earlier. After

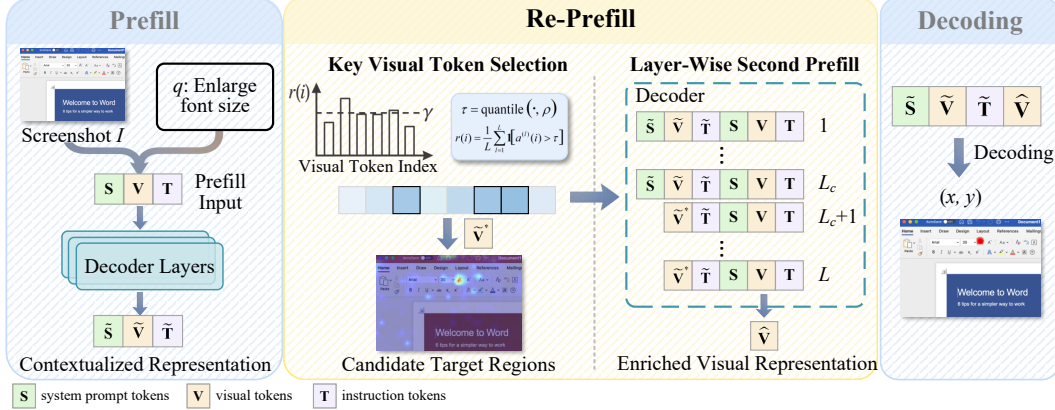


Figure 2: **Overview of Re-Prefill.** (1) *Prefill*. The input $[S; V; T]$ is processed through L decoder layers to obtain contextualized representations $[\tilde{S}; \tilde{V}; \tilde{T}]$. (2) *Key visual token selection*. Visual tokens that consistently receive high attention across layers are selected as $\tilde{V}^*$, representing candidate target regions. (3) *Layer-wise second prefill*. A copy of the original input is re-encoded with layer-wise prefix modification, transitioning from full first-prefill representations to key visual tokens, producing enriched visual representations $\hat{V}$. (4) *Decoding*. Coordinate tokens are generated autoregressively from the combined context $C^* = [\tilde{S}; \tilde{V}; \tilde{T}; \hat{V}]$.

$t = 1$, the trajectories diverge. The Correct group stabilizes near the target, whereas the Wrong group remains significantly farther away, with the gap persisting throughout decoding. This persistent discrepancy indicates that errors introduced during prefill are rarely corrected during decoding. This limitation arises from the autoregressive inference process, where decoding conditions on cached representations without recomputing attention over visual tokens. Consequently, misalignment formed during prefill propagates through all subsequent decoding steps.

These observations identify the prefill stage as a structural bottleneck in GUI grounding, as it determines target selection while its errors remain irreversible. Motivated by this finding, we propose to enable explicit re-evaluation of target selection before decoding, leading to the Re-Prefill method described in Section 3.

3 Re-Prefill

Building on the above analysis, we propose **Re-Prefill**, a training-free method that enhances GUI grounding through an attention-guided second prefill stage. Re-Prefill first identifies key visual tokens that encode candidate target regions from the initial prefill, and then performs a second prefill over the original input, where these tokens are incorporated as guidance to refine target region selection before decoding. An overview of the full pipeline is shown in Figure 2.

Step 1: Initial prefill and attention extraction. A standard prefill stage is first applied to the input $\mathbf{x} = [S; V; T]$, producing contextualized representations $\tilde{S}$, $\tilde{V}$, and $\tilde{T}$, together with the KV cache C_1 . To support subsequent visual token filtering, we additionally extract per-layer attention maps from the query position to visual tokens. Specifically, for each decoder layer l , the attention $\mathbf{a}^{(l)} \in \mathbb{R}^{N_v}$ is computed and averaged across heads to capture cross-layer visual relevance signals.

Step 2: Key visual token selection. The initial prefill produces contextualized visual representations $\tilde{V}$ and query-to-visual attention maps. Based on qualitative visualizations (Section 4.2 and Appendix C), we observe that the prefill-stage attention is often distributed across multiple plausible UI regions, while the region corresponding to the final prediction typically receives relatively high attention. This suggests that the first prefill implicitly narrows the search space to a set of candidate target regions before selecting one for subsequent decoding. In this process, visual tokens with relatively high attention act as coarse indicators of these candidate regions. Motivated by this observation, we seek to re-evaluate the candidate regions before decoding, thereby increasing the likelihood of selecting the correct target. This requires identifying a subset of contextualized visual tokens that reliably represent the candidate regions. To this end, we introduce a cross-layer consistency filter over L decoder layers. We first define a global high-attention threshold τ as the ρ -quantile over all

attention values:

$$\tau = \text{quantile}(\{a^{(l)}(i)\}_{l,i}, \rho). \quad (3)$$

For each visual token, we compute its cross-layer activation ratio:

$$r(i) = \frac{1}{L} \sum_{l=1}^L \mathbb{1}[a^{(l)}(i) > \tau]. \quad (4)$$

The *key visual tokens* are then defined as:

$$\tilde{\mathbf{V}}^* = \{ \tilde{v}_i \mid r(i) \geq \gamma \}, \quad (5)$$

which correspond to tokens that consistently exhibit high attention, encoding candidate target regions.

Step 3: Layer-wise second prefill. To enable refined target selection, we perform a second prefill over a copy of the original uncontextualized input $[\mathbf{S}; \mathbf{V}; \mathbf{T}]$, guided by the representations obtained from the first prefill.

The key design is to progressively transition from full-context conditioning to focused conditioning on key visual tokens. This is achieved through a layer-wise prefix injection strategy controlled by a continuity hyperparameter L_c , which determines the layer at which the prefix is updated. Specifically, for layers 1 to L_c , we prepend the complete first-prefill representations, resulting in $[\tilde{\mathbf{S}}; \tilde{\mathbf{V}}; \tilde{\mathbf{T}}; \mathbf{S}; \mathbf{V}; \mathbf{T}]$, so as to preserve global semantic alignment. For layers $L_c + 1$ to L , we replace the full visual representations with the selected key visual tokens and remove $\tilde{\mathbf{S}}$, yielding $[\tilde{\mathbf{V}}^*; \tilde{\mathbf{T}}; \mathbf{S}; \mathbf{V}; \mathbf{T}]$, so that the model focuses on candidate target regions while retaining instruction context. Since the instruction segment is typically short and contains essential information, no additional selection is applied. After all L layers, the visual tokens in the uncontextualized input are updated into enriched representations, denoted as $\hat{\mathbf{V}}$, which are subsequently used for decoding.

The hyperparameter L_c controls the trade-off between preserving semantic continuity and enabling effective re-focusing. Its impact is analyzed in Section 4.3.

Step 4: Decoding. The final stage produces coordinate tokens through autoregressive decoding. The decoding context $\mathcal{C}^*$ is constructed by concatenating the first-prefill representations with the enriched visual tokens $\hat{\mathbf{V}}$ from the second prefill, forming $\mathcal{C}^* = [\tilde{\mathbf{S}}; \tilde{\mathbf{V}}; \tilde{\mathbf{T}}; \hat{\mathbf{V}}]$. Coordinate tokens are then generated autoregressively over $\mathcal{C}^*$. Pseudocode of the full procedure is provided in Appendix G.

4 Experiments

4.1 Experimental Setup

Benchmarks. Re-Prefill is evaluated on five benchmarks spanning desktop, mobile, and web interfaces, including ScreenSpot-Pro [19], ScreenSpot-V2 [9], OSWorld-G [20], UI-Vision [21], and MMBench-GUI-L2 [22].

Base models. Four decoder-only VLMs are considered, including Qwen3-VL-8B-Instruct, Qwen3-VL-32B-Instruct [18], MAI-UI-8B [6], and GUI-Owl-1.5-8B-Instruct [7]. The two Qwen3-VL models are general-purpose VLMs without GUI-specific post-training, whereas MAI-UI-8B and GUI-Owl-1.5-8B-Instruct are designed specifically for GUI grounding. This selection covers the primary design configurations in current GUI agents and enables evaluation of whether Re-Prefill generalizes across different base model families.

Implementation details. Re-Prefill involves three hyperparameters, including the quantile threshold ρ , the ratio threshold γ , and the continuity layer L_c . The values $\rho = 0.8$ and $\gamma = 0.1$ are used throughout, with $L_c = 3$ for the 36-layer 8B models and $L_c = 6$ for the 64-layer 32B model. On ScreenSpot-Pro, OSWorld-G, UI-Vision, and MMBench-GUI-L2, a lightweight zoom-in strategy is applied to both the baseline and Re-Prefill. A square region centered on the initial prediction is cropped, upsampled by $2\times$, and then reprocessed by the same model to obtain a refined prediction. The zoom-in strategy is not applied on ScreenSpot-V2, as its low-resolution images already exhibit saturated performance without post-processing. The results for the base models are reproduced in this work under identical inference settings, whereas results for external baselines are taken from the original papers and are indicated accordingly in each table. We run 8B models on 8 RTX 4090 GPUs and the 32B model on 4 RTX PRO 6000 GPUs.

Table 1: Results on ScreenSpot-Pro. Methods marked with [†] are reproduced using the zoom-in strategy adopted in this work.

Model	Development		Creative		CAD		Scientific		Office		OS		Overall
	Text	Icon	Text	Icon	Text	Icon	Text	Icon	Text	Icon	Text	Icon	
UI-TARS-72B [23]	63.0	17.3	57.1	15.4	18.8	12.5	64.6	20.9	63.3	26.4	42.1	15.7	38.1
GTA1-32B [4]	83.1	37.9	72.2	25.9	70.1	31.3	84.7	39.1	89.3	64.2	76.6	51.7	63.6
UI-Ins-32B [24]	51.8	29.7	83.1	26.9	69.7	18.9	83.3	34.5	88.7	50.9	70.1	34.8	57.0
GuirVG [25]	64.9	7.6	42.9	11.2	28.9	9.4	63.9	16.4	63.8	26.4	43.9	13.5	36.1
GUI-Cursor-7B [26]	80.5	33.1	65.7	18.2	62.4	25.0	83.3	32.7	84.2	43.4	65.4	31.5	56.5
MAI-UI-8B [6]	78.6	58.6	78.8	46.9	80.7	43.8	86.1	49.1	88.1	81.1	76.6	51.7	70.9
GUI-Owl-1.5-8B-Instruct [7]	90.2	68.9	84.8	56.6	86.8	62.5	89.5	55.4	91.5	71.6	86.9	53.9	77.8
<i>Training-free methods</i>													
RegionFocus [12]	75.3	25.5	76.3	30.8	71.6	28.1	87.5	39.1	87.0	60.4	74.8	36.0	61.6
UI-Zoomer [15]	85.7	42.1	75.1	44.8	76.1	40.6	84.0	42.7	86.5	69.8	83.2	48.3	67.8
ZoomClick [13]	88.3	55.9	82.8	38.5	80.7	42.2	90.2	40.9	93.8	77.4	84.1	51.7	72.1
MVP-8B [16]	-	-	-	-	-	-	-	-	-	-	-	-	65.3
MVP-32B [16]	-	-	-	-	-	-	-	-	-	-	-	-	74.0
<i>Qwen3-VL-8B-Instruct [18][†]</i>	87.7	37.9	78.8	36.4	72.6	28.1	86.8	38.2	89.3	54.7	79.4	47.2	65.8
+ Re-Prefill	90.9	47.6	81.8	39.2	73.1	40.6	89.6	41.8	92.7	66.0	86.0	50.6	70.1 _{+4.3}
<i>MAI-UI-8B [6][†]</i>	83.8	49.0	77.3	40.6	79.2	34.4	85.4	48.2	91.5	69.8	80.4	56.2	69.6
+ Re-Prefill	87.7	56.6	80.3	44.8	78.7	45.3	81.9	50.9	91.5	77.4	83.2	55.1	72.0 _{+2.4}
<i>GUI-Owl-1.5-8B-Instruct [7][†]</i>	81.2	69.7	83.8	58.0	84.8	57.8	93.1	65.5	93.2	69.8	85.1	61.8	78.0
+ Re-Prefill	88.3	73.8	85.9	58.0	85.8	64.1	91.0	65.5	93.2	75.5	86.9	67.4	80.1 _{+2.1}
<i>Qwen3-VL-32B-Instruct [18][†]</i>	89.6	49.7	85.4	46.2	83.8	45.3	89.6	47.3	94.4	77.4	91.6	53.9	74.3
+ Re-Prefill	93.5	54.5	87.4	51.7	83.8	43.8	94.4	49.1	94.9	77.4	88.8	64.0	76.8 _{+2.5}

Table 2: Results on OSWorld-G. Methods marked with [†] are reproduced using the zoom-in strategy adopted in this work.

Model	Text Match.	Element Rec.	Layout Under.	Fine-grained Manip.	Refusal	Overall
UI-TARS-72B [23]	69.4	60.6	62.9	45.6	-	57.1
UI-TARS-1.5-7B [27]	36.8	62.7	62.2	50.8	-	52.8
GTA1-32B [4]	63.2	78.4	73.3	65.2	-	65.2
MAI-UI-8B [6]	72.8	67.6	71.1	56.4	-	64.2
GUI-Owl-1.5-8B-Instruct [7]	67.8	68.5	68.5	65.5	42.6	65.8
<i>Qwen3-VL-8B-Instruct [18][†]</i>	72.8	66.7	67.2	53.7	-	63.0
+ Re-Prefill	75.5	69.7	71.2	54.4	-	65.7 _{+2.7}
<i>MAI-UI-8B [6][†]</i>	75.5	70.3	70.0	53.7	-	65.5
+ Re-Prefill	76.6	71.8	74.7	55.7	-	67.7 _{+2.2}
<i>GUI-Owl-1.5-8B-Instruct [7][†]</i>	75.5	71.8	73.5	56.4	-	67.2
+ Re-Prefill	77.4	73.0	74.3	59.7	-	68.8 _{+1.6}
<i>Qwen3-VL-32B-Instruct [18][†]</i>	76.6	73.9	77.9	54.4	-	69.0
+ Re-Prefill	78.9	74.8	77.5	57.0	-	70.1 _{+1.1}

4.2 Main Results

Tables 1–5 report results on the five benchmarks. The two MVP variants [16] share base models with our evaluation (MVP-8B uses Qwen3-VL-8B-Instruct and MVP-32B uses Qwen3-VL-32B-Instruct), enabling a direct comparison of inference-time strategies under identical underlying VLMs.

High-resolution and dense UI grounding. ScreenSpot-Pro evaluates grounding on high-resolution interfaces with dense layouts. As shown in Table 1, Re-Prefill consistently improves all four models, with gains of +4.3 (Qwen3-VL-8B), +2.4 (MAI-UI-8B), +2.1 (GUI-Owl-8B), and +2.5 (Qwen3-VL-32B). For Qwen3-VL models, performance increases from 65.8% to 70.1% (8B) and from 74.3% to 76.8% (32B), outperforming MVP (65.3% and 74.0%) under the same backbone. For GUI-specialized models, Re-Prefill also yields consistent gains, improving MAI-UI-8B from 69.6% to 72.0% and GUI-Owl-1.5-8B from 78.0% to 80.1%, despite prior GUI-specific training. These results show that Re-Prefill complements training-based approaches by refining target selection at inference time. The larger gains on general-purpose models suggest that improving the prefill stage is particularly beneficial when initial target selection is less reliable.

Complex desktop interaction grounding. OSWorld-G evaluates grounding in desktop environments requiring structural UI understanding and interaction-aware reasoning. It provides original and refined annotation versions. This section reports results on the original annotations, while the refined results are provided in Appendix E. As shown in Table 2, Re-Prefill improves all models, with gains of +2.7

Table 3: Results on ScreenSpot-V2. Methods marked with [†] are reproduced in this work.

Model	Mobile		Desktop		Web		Overall
	Text	Icon	Text	Icon	Text	Icon	
UI-TARS-72B [23]	94.8	86.3	91.2	87.9	91.5	87.7	90.3
GTA1-32B [4]	99.7	90.5	99.0	94.3	95.7	90.1	95.2
GUI-Cursor-7B [26]	99.2	90.6	94.4	91.3	96.1	89.0	93.9
UI-Ins-32B [24]	98.6	90.0	99.0	87.9	97.0	93.1	94.9
GuirIVG [25]	98.3	89.6	94.3	80.7	95.7	86.2	91.9
GUI-Owl-1.5-8B-Instruct [7]	97.4	90.5	96.4	90.7	94.2	89.7	93.7
<i>Qwen3-VL-8B-Instruct [18][†]</i>	99.3	88.2	97.9	87.9	96.6	87.2	93.6
+ Re-Prefill	99.7	89.6	97.9	90.0	97.0	90.1	94.7 _{+1.1}
<i>MAI-UI-8B [6][†]</i>	99.0	89.6	97.9	91.4	97.4	91.6	94.2
+ Re-Prefill	99.3	90.0	97.9	92.1	97.9	91.6	95.3 _{+1.1}
<i>GUI-Owl-1.5-8B-Instruct [7][†]</i>	98.6	90.0	97.4	89.3	96.2	88.2	93.9
+ Re-Prefill	98.3	91.0	97.9	92.1	96.2	89.7	94.6 _{+0.7}
<i>Qwen3-VL-32B-Instruct [18][†]</i>	99.0	91.5	98.5	90.0	96.6	91.1	95.0
+ Re-Prefill	99.3	91.9	98.5	91.4	97.4	90.1	95.3 _{+0.3}

(Qwen3-VL-8B), +2.2 (MAI-UI-8B), +1.6 (GUI-Owl-8B), and +1.1 (Qwen3-VL-32B). Compared with ScreenSpot-Pro, improvements are smaller but more stable across model scales. Qwen3-VL improves from 63.0% to 65.7% (8B) and from 69.0% to 70.1% (32B), while MAI-UI-8B and GUI-Owl-1.5-8B increase from 65.5% to 67.7% and from 67.2% to 68.8%, respectively. These results indicate that Re-Prefill remains effective under complex interaction settings, improving both visual discrimination and structural reasoning. The smaller gains suggest that when grounding relies more on interaction structure than visual ambiguity, the benefit of prefill refinement becomes less pronounced but remains consistent.

Cross-platform UI grounding. ScreenSpot-V2 evaluates grounding across mobile, desktop, and web interfaces. As shown in Table 3, Re-Prefill improves all models, with gains of +1.1 (Qwen3-VL-8B), +1.1 (MAI-UI-8B), +0.7 (GUI-Owl-8B), and +0.3 (Qwen3-VL-32B). Improvements are smaller than on ScreenSpot-Pro and OSWorld-G due to the already high baseline performance (above 93%), leaving limited room for improvement. Despite this saturation, gains remain consistent across model types and platforms, including both mobile and desktop interfaces. These results suggest that Re-Prefill generalizes well across diverse UI domains, acting as a robust inference-time enhancement even when performance is near saturation.

Instruction-diverse grounding. We further evaluate Re-Prefill on instruction-heavy grounding scenarios using UI-Vision and MMBench-GUI-L2, which require robust query understanding under varying levels of semantic abstraction. On UI-Vision (Table 4), Re-Prefill improves all models, with gains of +1.0 (Qwen3-VL-8B), +1.3 (MAI-UI-8B), +0.4 (GUI-Owl-8B), and +0.9 (Qwen3-VL-32B). On MMBench-GUI-L2 (Table 5), the gains increase to +2.2, +1.3, +0.6, and +1.1, respectively. The consistent improvements across both benchmarks indicate that Re-Prefill remains effective under diverse instructions. Compared with scenarios dominated by visual ambiguity or structural complexity, these results suggest that refining the prefill stage also benefits semantic alignment between instructions and UI elements, enabling more robust grounding under varying query formulations.

Table 4: Results on UI-Vision. Methods marked with [†] are reproduced using the zoom-in strategy adopted in this work.

Model	Basic	Functional	Spatial	Overall
UI-TARS-72B [23]	31.4	30.5	14.7	25.5
UI-TARS-1.5-7B [27]	28.8	27.5	10.7	22.3
MAI-UI-8B [6]	51.6	50.5	26.6	42.4
<i>Training-free methods</i>				
UI-Zoomer [15]	42.5	40.1	19.7	33.7
ZoomClick [13]	45.1	45.1	28.3	39.2
MVP-8B [16]	38.3	38.8	18.7	31.9
MVP-32B [16]	49.4	52.0	30.8	44.1
<i>Qwen3-VL-8B-Instruct [18][†]</i>	38.2	39.9	19.3	32.1
+ Re-Prefill	39.4	41.7	19.3	33.1 _{+1.0}
<i>MAI-UI-8B [6][†]</i>	50.7	50.4	27.0	42.2
+ Re-Prefill	52.3	52.8	27.0	43.5 _{+1.3}
<i>GUI-Owl-1.5-8B-Instruct [7][†]</i>	50.1	44.7	29.8	41.2
+ Re-Prefill	50.1	46.1	29.7	41.6 _{+0.4}
<i>Qwen3-VL-32B-Instruct [18][†]</i>	49.9	51.1	31.1	43.7
+ Re-Prefill	51.4	51.0	32.4	44.6 _{+0.9}

Qualitative analysis. Figure 3 visualizes query-position attention across stages for a representative

Table 5: Results on MMBench-GUI-L2. Methods marked with $\dagger$ are reproduced using the zoom-in strategy adopted in this work.

Model	Windows		MacOS		Linux		iOS		Android		Web		Overall
	Basic	Adv.	Basic	Adv.	Basic	Adv.	Basic	Adv.	Basic	Adv.	Basic	Adv.	
UI-TARS-72B [23]	78.6	51.8	80.3	62.7	68.6	51.5	90.8	81.2	93.0	80.0	88.1	68.5	74.3
UI-TARS-1.5-7B [27]	68.3	39.0	69.0	44.5	64.4	37.8	88.5	69.4	90.5	69.3	81.0	56.5	64.3
UI-Ins-32B [24]	84.9	68.4	88.4	73.4	68.6	56.1	96.5	91.2	97.2	92.4	94.8	85.1	84.9
MAI-UI-8B [6]	92.3	74.3	90.7	86.4	81.2	67.3	97.1	90.0	97.5	92.7	95.8	86.0	88.8
GUI-Owl-1.5-8B-Instruct [7]	89.7	65.4	88.1	72.8	72.8	56.6	95.9	83.9	95.2	82.9	93.2	77.6	82.5
<i>Qwen3-VL-8B-Instruct [18]$\dagger$</i>	90.4	70.6	86.7	73.4	77.5	62.8	95.2	85.8	95.5	86.8	94.8	78.2	84.2
<i>+ Re-Prefill</i>	90.0	73.5	89.0	78.3	82.2	67.9	96.2	86.7	96.1	88.5	95.5	82.1	86.4 ± 2.2
<i>MAI-UI-8B [6]$\dagger$</i>	93.0	71.0	91.9	78.6	83.2	67.9	96.5	87.6	98.0	88.2	96.4	83.1	87.2
<i>+ Re-Prefill</i>	93.0	74.6	93.0	80.6	83.8	71.4	95.9	90.0	97.8	91.3	95.5	84.4	88.5 ± 1.3
<i>GUI-Owl-1.5-8B-Instruct [7]$\dagger$</i>	92.6	65.8	88.4	73.1	82.2	58.2	97.1	86.4	97.5	83.9	95.8	80.2	84.5
<i>+ Re-Prefill</i>	90.8	68.8	87.8	74.3	82.7	61.7	96.5	86.7	97.8	85.1	95.5	81.2	85.1 ± 0.6
<i>Qwen3-VL-32B-Instruct [18]$\dagger$</i>	93.4	71.3	90.4	80.3	86.9	70.4	97.1	90.3	97.8	91.5	96.8	84.1	88.4
<i>+ Re-Prefill</i>	92.6	72.4	91.9	84.7	85.9	76.0	97.5	91.8	96.9	91.5	97.1	86.4	89.5 ± 1.1

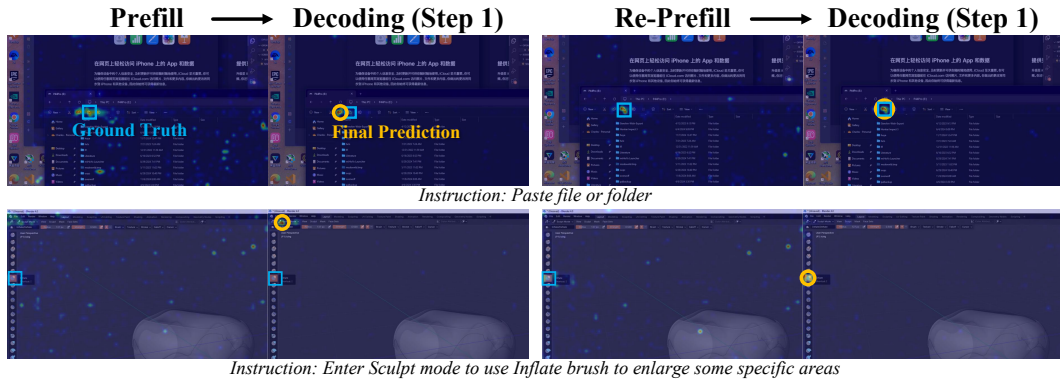


Figure 3: **Query-position attention heatmaps across stages on ScreenSpot-Pro.** The first two panels illustrate the baseline transition, while the last two panels show the corresponding transition under Re-Prefill. The blue rectangle marks the ground-truth target, and the orange circle indicates the predicted coordinate. Re-Prefill focuses attention on the correct region during prefill, suppresses distractors, and leads to more accurate localization in subsequent decoding.

example, comparing the baseline with Re-Prefill. In the baseline, attention at prefill is dispersed over multiple candidate regions and may contract onto an incorrect region during decoding. In contrast, Re-Prefill produces more focused attention on the correct target during prefill, leading to more accurate localization in subsequent decoding. This observation aligns with the two-stage inference structure in Section 2.2, where target selection is determined during prefill and propagated through decoding. Additional qualitative results are provided in Appendix C.

Overall, Re-Prefill consistently improves grounding accuracy across diverse settings. A detailed analysis of computational efficiency is provided in Appendix F.

4.3 Ablation Studies

Ablation studies are conducted on ScreenSpot-Pro with Qwen3-VL-8B-Instruct as the base model.

Analysis of the Re-Prefill mechanism. Three variants are designed to verify the contribution of the Re-Prefill mechanism (Table 6). *Embedding Addition* reuses the first-prefill visual tokens $\tilde{V}$ in the decoding context instead of the enriched $\hat{V}$, thereby skipping the second prefill stage. *Blind Re-Prefill* omits key-token selection and appends the full contextualized visual tokens $\tilde{V}$ in the second prefill. *Random Token Selection* replaces attention-

Table 6: Analysis of Re-Prefill mechanism. Embedding Addition removes the second prefill and uses the first-prefill visual tokens in decoding. Blind Re-Prefill omits selection and uses all visual tokens. Random Token Selection replaces attention-based selection with random sampling.

Method	Acc.
Baseline	65.8
Embedding Addition	66.0
Blind Re-Prefill	69.4
Random Token Selection	69.1
Re-Prefill (Ours)	70.1

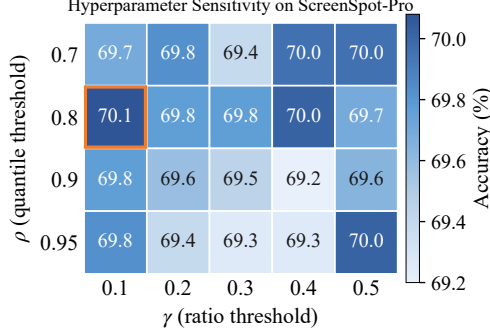


Figure 4: Effect of ρ and γ . The orange box marks the default configuration ($\rho=0.8$, $\gamma=0.1$). All twenty cells exceed the baseline (65.8%) with a spread of less than one point, indicating that Re-Prefill is largely insensitive to these thresholds.

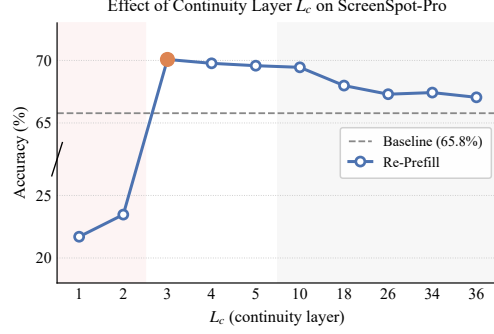


Figure 5: Effect of L_c . The optimum at $L_c=3$ balances two modes. For small L_c , insufficient semantic alignment arises between uncontextualized input tokens and the first-prefill prefix (red zone). For large L_c , noise from unrelated tokens propagates into deeper layers (grey zone).

based token selection with random sampling. All three variants perform worse than Re-Prefill. The largest gap, observed for Embedding Addition (66.0%), confirms that the primary source of improvement is the second prefill stage. The benefit arises from allowing the original visual tokens to undergo a second prefill guided by contextualized key visual tokens and instruction tokens, rather than merely emphasizing visual information from the first pass. The gap to Blind Re-Prefill (69.4%) indicates that focusing the second pass on a compact set of key visual tokens is more effective than propagating the full sequence. The gap to Random Token Selection (69.1%) shows that the attention-derived selection signal further enhance performance by filtering out unrelated tokens and reducing interference from noisy information.

Analysis of hyperparameters ρ and γ . The selection of $\tilde{\mathbf{V}}^*$ depends on the quantile threshold ρ (Eq. 3) and the ratio threshold γ (Eq. 5). Figure 4 presents a 4×5 grid over $\rho \in \{0.70, 0.80, 0.90, 0.95\}$ and $\gamma \in \{0.1, 0.2, 0.3, 0.4, 0.5\}$. All twenty configurations outperform the 65.8% baseline, and the variation across the grid remains within 1.0 point (69.2% to 70.1%), indicating that Re-Prefill is largely insensitive to these thresholds. The best configuration ($\rho = 0.8$, $\gamma = 0.1$) lies at the more permissive end of both axes, expanding $\tilde{\mathbf{V}}^*$ to include not only the most highly attended region but also neighboring candidates that compete for attention. This design allows the second prefill stage to recover when the initial selection corresponds to an incorrect UI element. This configuration is adopted in all main results.

Analysis of continuity layers L_c . L_c determines how many early decoder layers retain the full prefix from the first prefill before it is reduced to the key visual tokens $\tilde{\mathbf{V}}^*$ and instruction tokens $\tilde{\mathbf{T}}$. A sweep of L_c from 1 to 36 is conducted on Qwen3-VL-8B-Instruct ($L = 36$). Figure 5 shows that the accuracy drops to approximately 22% for $L_c \in \{1, 2\}$, reaches a maximum of 70.1% at $L_c = 3$, and then decreases monotonically. This behavior reflects the dual role of L_c . Early layers ($\leq L_c$) use the full contextualized representations from the first prefill as a prefix to align the copy of uncontextualized input tokens with it. Later layers ($> L_c$) retain only $\tilde{\mathbf{V}}^*$ and $\tilde{\mathbf{T}}$ so that subsequent processing concentrates on candidate regions without interference from unrelated tokens. When L_c is small, the prefix is truncated before semantic alignment is established, causing later layers to operate on uncalibrated inputs with incomplete cues, which leads to severe performance degradation. When L_c is large, unrelated tokens propagate into deeper layers and dilute attention away from candidate regions, thereby reducing accuracy. The optimal value at $L_c = 3$ balances these two failure modes.

5 Conclusion

This work analyzes the inference dynamics of decoder-only VLMs for GUI grounding and identifies a two-stage paradigm in which target selection is completed during the prefill stage, while decoding only refines coordinates within the selected region. The prefill stage therefore constitutes the decisive factor for grounding accuracy, yet it remains unaddressed by existing multi-run methods. Motivated by this observation, Re-Prefill is proposed as a training-free approach that calibrates the prefill

stage through attention-guided re-evaluation, in which key visual tokens and instruction tokens are reintroduced for a second prefill stage prior to coordinate decoding. Across four VLMs and five benchmarks, consistent improvements are achieved without additional training, reaching gains of up to 4.3 points on ScreenSpot-Pro and 2.7 points on OSWorld-G. These results establish prefill calibration as a principled direction for inference-time computation in vision-language models.

References

- [1] Dang Nguyen, Jian Chen, Yu Wang, Gang Wu, Namyong Park, Zhengmian Hu, Hanjia Lyu, Junda Wu, Ryan Aponte, Yu Xia, et al. Gui agents: A survey. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 22522–22538, 2025.
- [2] Chaoyun Zhang, Shilin He, Jiaxu Qian, Bowen Li, Liqun Li, Si Qin, Yu Kang, Minghua Ma, Guyue Liu, Qingwei Lin, et al. Large language model-brained gui agents: A survey. *arXiv preprint arXiv:2411.18279*, 2024.
- [3] Shuai Wang, Weiwen Liu, Jingxuan Chen, Yuqi Zhou, Weinan Gan, Xingshan Zeng, Yuhan Che, Shuai Yu, Xinlong Hao, Kun Shao, et al. Gui agents with foundation models: A comprehensive survey. *arXiv preprint arXiv:2411.04890*, 2024.
- [4] Yan Yang, Dongxu Li, Yutong Dai, Yuhao Yang, Ziyang Luo, Zirui Zhao, Zhiyuan Hu, Junzhe Huang, Amrita Saha, Zeyuan Chen, Ran Xu, Liyuan Pan, Caiming Xiong, and Junnan Li. GTAI: GUI test-time scaling agent. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=3VIPmz7iAi>.
- [5] Fei Tang, Zhangxuan Gu, Zhengxi Lu, Xuyang Liu, Shuheng Shen, Changhua Meng, Wen Wang, Wenqi Zhang, Yongliang Shen, Weiming Lu, et al. Gui-g²: Gaussian reward modeling for gui grounding. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pages 33214–33222, 2026.
- [6] Hanzhang Zhou, Xu Zhang, Panrong Tong, Jianan Zhang, Liangyu Chen, Quyu Kong, Chenglin Cai, Chen Liu, Yue Wang, Jingren Zhou, et al. Mai-ui technical report: Real-world centric foundation gui agents. *arXiv preprint arXiv:2512.22047*, 2025.
- [7] Haiyang Xu, Xi Zhang, Haowei Liu, Junyang Wang, Zhaozai Zhu, Shengjie Zhou, Xuhao Hu, Feiyu Gao, Junjie Cao, Zihua Wang, et al. Mobile-agent-v3. 5: Multi-platform fundamental gui agents. *arXiv preprint arXiv:2602.16855*, 2026.
- [8] Wenyi Hong, Weihang Wang, Qingsong Lv, Jiazheng Xu, Wenmeng Yu, Junhui Ji, Yan Wang, Zihan Wang, Yuxiao Dong, Ming Ding, et al. Cogagent: A visual language model for gui agents. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 14281–14290, 2024.
- [9] Zhiyong Wu, Zhenyu Wu, Fangzhi Xu, Yian Wang, Qiushi Sun, Chengyou Jia, Kanzhi Cheng, Zichen Ding, Liheng Chen, Paul Pu Liang, and Yu Qiao. OS-ATLAS: Foundation action model for generalist GUI agents. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=n9PDaFNi8t>.
- [10] Boyu Gou, Ruohan Wang, Boyuan Zheng, Yanan Xie, Cheng Chang, Yiheng Shu, Huan Sun, and Yu Su. Navigating the digital world as humans do: Universal visual grounding for GUI agents. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=kxnoqaisCT>.
- [11] Hyunseok Lee, Jeonghoon Kim, Beomjun Kim, Jihoon Tack, Chansong Jo, Jaehong Lee, Cheonbok Park, Sookyo In, Jinwoo Shin, and Kang Min Yoo. Reguide: Data efficient gui grounding via spatial reasoning and search. *arXiv preprint arXiv:2505.15259*, 2025.
- [12] Tiange Luo, Lajanugen Logeswaran, Justin Johnson, and Honglak Lee. Visual test-time scaling for gui agent grounding. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 19989–19998, 2025.

- [13] Zhiyuan Jiang, Shenghao Xie, Wenyi Li, Wenqiang Zu, Peihang Li, Jiahao Qiu, Siqi Pei, Lei Ma, Tiejun Huang, Mengdi Wang, et al. Zoom in, click out: Unlocking and evaluating the potential of zooming for gui grounding. *arXiv preprint arXiv:2512.05941*, 2025.
- [14] Aiden Yiliu Li, Bizhi Yu, Daoan Lei, Tianhe Ren, and Shilong Liu. Chain-of-ground: Improving gui grounding via iterative reasoning and reference feedback. *arXiv preprint arXiv:2512.01979*, 2025.
- [15] Fei Tang, Bofan Chen, Zhengxi Lu, Tongbo Chen, Songqin Nong, Tao Jiang, Wenhao Xu, Weiming Lu, Jun Xiao, Yueting Zhuang, et al. Ui-zoomer: Uncertainty-driven adaptive zoom-in for gui grounding. *arXiv preprint arXiv:2604.14113*, 2026.
- [16] Yunzhu Zhang, Zeyu Pan, Zhengwen Zeng, Shuheng Shen, Changhua Meng, and Linchao Zhu. Mvp: Multiple view prediction improves gui grounding. *arXiv preprint arXiv:2512.08529*, 2025.
- [17] Hang Wu, Hongkai Chen, Yujun Cai, Chang Liu, Qingwen Ye, Ming-Hsuan Yang, and Yiwei Wang. Dimo-gui: Advancing test-time scaling in gui grounding via modality-aware visual reasoning. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 26257–26267, 2025.
- [18] Shuai Bai, Yuxuan Cai, Ruizhe Chen, Keqin Chen, Xionghui Chen, Zesen Cheng, Lianghao Deng, Wei Ding, Chang Gao, Chunjiang Ge, et al. Qwen3-vl technical report. *arXiv preprint arXiv:2511.21631*, 2025.
- [19] Kaixin Li, Ziyang Meng, Hongzhan Lin, Ziyang Luo, Yuchen Tian, Jing Ma, Zhiyong Huang, and Tat-Seng Chua. Screenspot-pro: Gui grounding for professional high-resolution computer use. In *Proceedings of the 33rd ACM International Conference on Multimedia*, pages 8778–8786, 2025.
- [20] Tianbao Xie, Jiaqi Deng, Xiaochuan Li, Junlin Yang, Haoyuan Wu, Jixuan Chen, Wenjing Hu, Xinyuan Wang, Yuhui Xu, Zekun Wang, et al. Scaling computer-use grounding via user interface decomposition and synthesis. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2025.
- [21] Shravan Nayak, Xiangru Jian, Kevin Qinghong Lin, Juan A Rodriguez, Montek Kalsi, Rabiul Awal, Nicolas Chapados, M Tamer Özsu, Aishwarya Agrawal, David Vazquez, et al. Ui-vision: A desktop-centric gui benchmark for visual perception and interaction. *arXiv preprint arXiv:2503.15661*, 2025.
- [22] Xuehui Wang, Zhenyu Wu, JingJing Xie, Zichen Ding, Bowen Yang, Zehao Li, Zhaoyang Liu, Qingyun Li, Xuan Dong, Zhe Chen, et al. Mmbench-gui: Hierarchical multi-platform evaluation framework for gui agents. *arXiv preprint arXiv:2507.19478*, 2025.
- [23] Yujia Qin, Yining Ye, Junjie Fang, Haoming Wang, Shihao Liang, Shizuo Tian, Junda Zhang, Jiahao Li, Yunxin Li, Shijue Huang, et al. Ui-tars: Pioneering automated gui interaction with native agents. *arXiv preprint arXiv:2501.12326*, 2025.
- [24] Liangyu Chen, Hanzhang Zhou, Chenglin Cai, Jianan Zhang, Panrong Tong, Xu Zhang, Quyu Kong, Chen Liu, Yuqi Liu, Wenxuan Wang, Yue Wang, Qin Jin, and Steven HOI. UI-ins: Enhancing GUI grounding with multi-perspective instruction as reasoning. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=dsQHm7YX9c>.
- [25] Weitai Kang, Bin Lei, Gaowen Liu, Caiwen Ding, and Yan Yan. GuirVG: Incentivize GUI visual grounding via empirical exploration on reinforcement learning. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=zrH2A1upAo>.
- [26] Yu Zhao, Wei-Ning Chen, Huseyin Atahan Inan, Samuel Kessler, Lu Wang, Lukas Wutschitz, Fangkai Yang, Chaoyun Zhang, Pasquale Minervini, Saravan Rajmohan, et al. Learning gui grounding with spatial reasoning from visual feedback. *arXiv preprint arXiv:2509.21552*, 2025.

- [27] ByteDance Seed Team. Ui-tars-1.5: A multimodal ui understanding and reasoning model, 2025. URL <https://seed-tars.com/1.5>.
- [28] Fei Tang, HaoLei Xu, Hang Zhang, Siqi Chen, Xingyu Wu, Yongliang Shen, Wenqi Zhang, Guiyang Hou, Zeqi Tan, Yuchen Yan, et al. A survey on (m) llm-based gui agents. *arXiv preprint arXiv:2504.13865*, 2025.
- [29] Kevin Qinghong Lin, Linjie Li, Difei Gao, Zhengyuan Yang, Zechen Bai, Weixian Lei, Lijuan Wang, and Mike Zheng Shou. Showui: One vision-language-action model for generalist gui agent. In *NeurIPS 2024 Workshop on Open-World Agents*, 2024.
- [30] Kanzhi Cheng, Qiushi Sun, Yougang Chu, Fangzhi Xu, Li YanTao, Jianbing Zhang, and Zhiyong Wu. Seedclick: Harnessing gui grounding for advanced visual gui agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 9313–9332, 2024.
- [31] Yuhao Yang, Yue Wang, Dongxu Li, Ziyang Luo, Bei Chen, Chao Huang, and Junnan Li. Aria-ui: Visual grounding for gui instructions. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 22418–22433, 2025.
- [32] Taofeng Xue, Chong Peng, Mianqiu Huang, Linsen Guo, Tiancheng Han, Haozhe Wang, Jianing Wang, Xiaocheng Zhang, Xin Yang, Dengchang Zhao, et al. Evocua: Evolving computer use agents via learning from scalable synthetic experience. *arXiv preprint arXiv:2601.15876*, 2026.
- [33] Xinyuan Wang, Bowen Wang, Dunjie Lu, Junlin Yang, Tianbao Xie, Junli Wang, Jiaqi Deng, Xiaole Guo, Yiheng Xu, Chen Henry Wu, et al. Opencua: Open foundations for computer-use agents. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025.
- [34] Run Luo, Lu Wang, Wanwei He, Longze Chen, Jiaming Li, and Xiaobo Xia. Gui-r1: A generalist r1-style vision-language action model for gui agents. *arXiv preprint arXiv:2504.10458*, 2025.
- [35] Xinbin Yuan, Jian Zhang, Kaixin Li, Zhuoxuan Cai, Lujian Yao, Jie Chen, Enguang Wang, Qibin Hou, Jinwei Chen, Peng-Tao Jiang, et al. Enhancing visual grounding for gui agents via self-evolutionary reinforcement learning. *arXiv preprint arXiv:2505.12370*, 2025.
- [36] Zhangxuan Gu, Zhengwen Zeng, Zhenyu Xu, Xingran Zhou, Shuheng Shen, Yunfei Liu, Beitong Zhou, Changhua Meng, Tianyu Xia, Weizhi Chen, et al. Ui-venus technical report: Building high-performance ui agents with rft. *arXiv preprint arXiv:2508.10833*, 2025.
- [37] Chen Chen, Jiawei Shao, Dakuan Lu, Haoyi Hu, Xiangcheng Liu, Hantao Yao, and Wu Liu. Gui-eyes: Tool-augmented perception for visual grounding in gui agents. *arXiv preprint arXiv:2601.09770*, 2026.
- [38] Charlie Victor Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling LLM test-time compute optimally can be more effective than scaling parameters for reasoning. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=4FWAwZtd2n>.
- [39] Bin Lei, Nuo Xu, Ali Payani, Mingyi Hong, Chunhua Liao, Yu Cao, and Caiwen Ding. Gui-spotlight: Adaptive iterative focus refinement for enhanced gui visual grounding. *arXiv preprint arXiv:2510.04039*, 2025.
- [40] Anthony Nguyen. Improved gui grounding via iterative narrowing. *arXiv preprint arXiv:2411.13591*, 2024.
- [41] Yong Du, Yuchen Yan, Fei Tang, Zhengxi Lu, Chang Zong, Weiming Lu, Shengpei Jiang, and Yongliang Shen. Test-time reinforcement learning for gui grounding via region consistency. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pages 30593–30601, 2026.

A Broader Impact

This work proposes a training-free method for improving GUI grounding in vision-language models. By improving grounding accuracy without additional training, the method may benefit applications such as desktop automation, accessibility tools, and resource-efficient GUI agents. In addition, our analysis of prefill-stage behavior may encourage further research on more interpretable and reliable multimodal inference. At the same time, improved GUI grounding may increase the capability of autonomous agents to interact with software environments, which could potentially be misused for unintended or unauthorized automation.

B Related Work

B.1 GUI Grounding

GUI grounding is a core capability for autonomous GUI agents [1–3, 28]. Early approaches address this task through supervised fine-tuning on grounding-specific datasets [8, 10, 29]. SeeClick [30] formulates GUI grounding as a standalone pre-training objective and demonstrates that task-specific training improves agent performance. Subsequent work extends this direction by introducing more diverse instruction types and cross-platform training data [9, 10, 31–33]. More recent studies apply reinforcement learning to GUI grounding [34–37]. These methods optimize task-level metrics and learn from interaction feedback. GUI-Cursor [26] models grounding as an iterative cursor movement process, where predictions are refined through visual feedback. Other approaches improve grounding through multi-perspective instruction reasoning [24], stabilized reward optimization [25], and joint planning with grounding in agent frameworks [4]. Despite strong performance, both supervised and reinforcement learning approaches require substantial training data and computational resources.

B.2 Inference-Time Enhancement for GUI Grounding

Inference-time scaling improves performance without additional training [38]. In GUI grounding, one line of work focuses on zoom-in-based inference. These methods iteratively crop and zoom into candidate regions, increasing resolution and reducing the search space [12–14, 39, 40]. For example, ZoomClick [13] identifies the essential characteristics of zooming and enhances target-focused resolution through a repeated crop-and-zoom strategy. RegionFocus [12] apply dynamic zoom-in strategy to reduce background interference, thus enhancing grounding accuracy. Another line improves grounding accuracy by aggregating predictions across multiple runs [11, 15–17, 41]. For instance, MVP [16] aggregates predictions from multiple cropped views to reduce variance. DiMo-GUI [17] reasons over textual and iconic elements separately and selects the best candidate based on global analysis. UI-Zoomer [15] samples multiple candidate predictions and uses consensus voting together with adaptive cropping to achieve more robust grounding. Despite their differences, these approaches share a common limitation. They do not modify the internal forward process of the model. Instead, they vary the input through iterative zoom-and-crop operations or aggregate outputs from multiple inferences. As a result, each forward pass still contains a single prefill stage, and target selection must be completed in one attempt.

C Additional Qualitative Analysis

This appendix presents qualitative visualizations of query-position attention over visual tokens. For each example, we compare the baseline transition from prefill to the first decoding step with the corresponding transition under Re-Prefill, where a second prefill is applied before decoding. All results are produced using Qwen3-VL-8B-Instruct on ScreenSpot-Pro. Ground-truth targets are marked with blue rectangles, and predicted coordinates are shown as orange hollow circles for visibility.

Prefill vs. Re-Prefill. Under the baseline, attention at the prefill stage is distributed across multiple candidate regions, consistent with the high variance observed in Figure 1(b). While these regions often include the ground-truth target, distractors with similar appearance or semantics are also activated, making disambiguation difficult in a single forward pass. With Re-Prefill, attention becomes more concentrated on the correct region, while responses on distractor regions are noticeably suppressed.

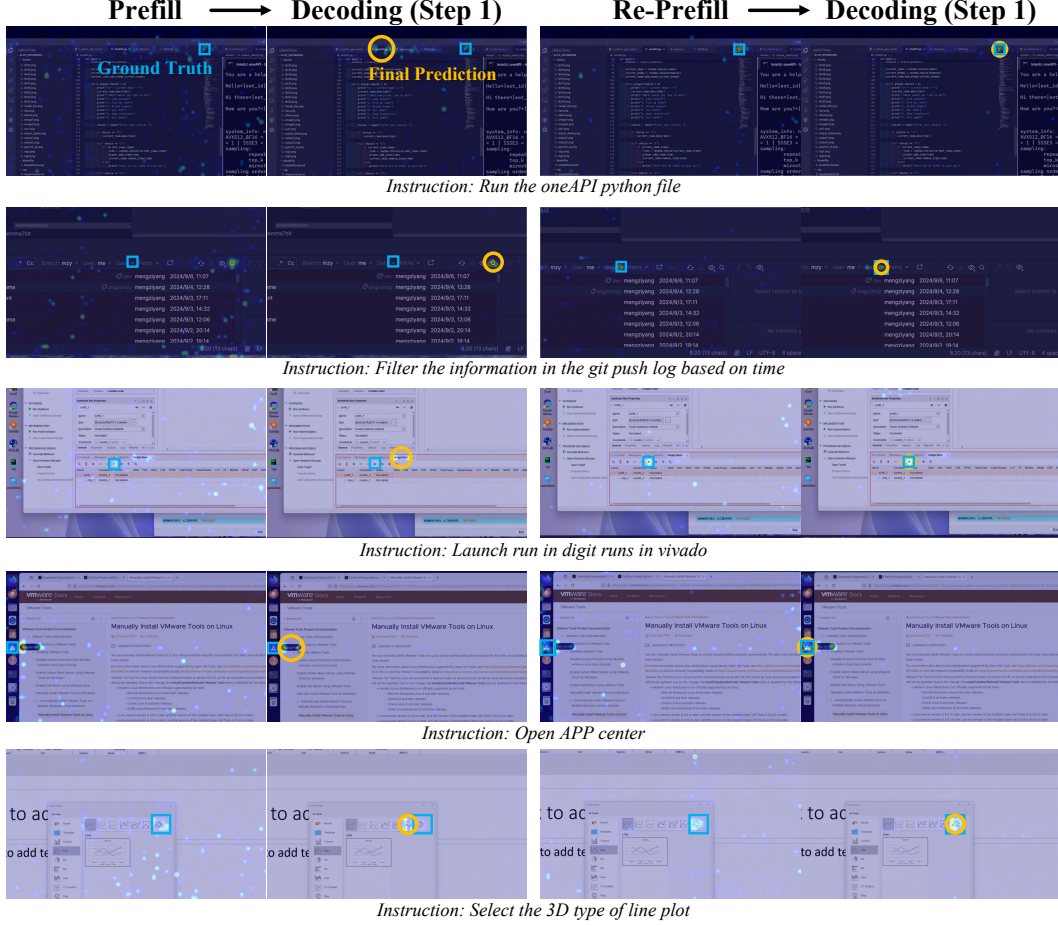


Figure 6: **Query-position attention heatmaps across stages on ScreenSpot-Pro.** Each row shows one example. Columns 1–2 present the baseline transition from prefill to the first decoding step, while Columns 3–4 show the corresponding transition with Re-Prefill. The blue rectangle marks the ground-truth target, and the orange circle indicates the predicted coordinate. Compared to the baseline, Re-Prefill focuses attention on the correct region during prefill, suppresses distractors, and improves localization for subsequent decoding.

As a result, the effective candidate set is reduced, leading to more reliable target selection before decoding. This supports the design of Re-Prefill, where key visual tokens extracted from the first prefill guide a second prefill to refine attention within candidate regions and improve target selection.

The first decoding step. At the first decoding step, attention contracts sharply to a localized region in both settings, consistent with the drop in spatial variance from $t = 0$ to $t = 1$ in Figure 1(b). The predicted coordinate is placed within this region, indicating that target selection has already been determined before decoding. With Re-Prefill, this localized region is more likely to align with the ground-truth target, whereas the baseline may converge to a distractor region. This demonstrates that errors introduced during prefill persist into decoding, while improvements from Re-Prefill are directly propagated to coordinate prediction, consistent with the prefill bottleneck identified in Section 2.2.

D Additional Analysis of Prefill-Stage Attention Dynamics

This appendix extends the analysis in Section 2.2 to multiple models and benchmarks. Experiments are conducted on two VLMs, Qwen3-VL-8B-Instruct and GUI-Owl-1.5-8B-Instruct, across three datasets, ScreenSpot-Pro, OSWorld-G, and MMBench-GUI. Figure 7 summarizes the results. The first two rows correspond to Qwen3-VL-8B-Instruct, and the last two rows correspond to GUI-Owl-1.5-8B-Instruct. The columns correspond to ScreenSpot-Pro, OSWorld-G, and MMBench-GUI. For

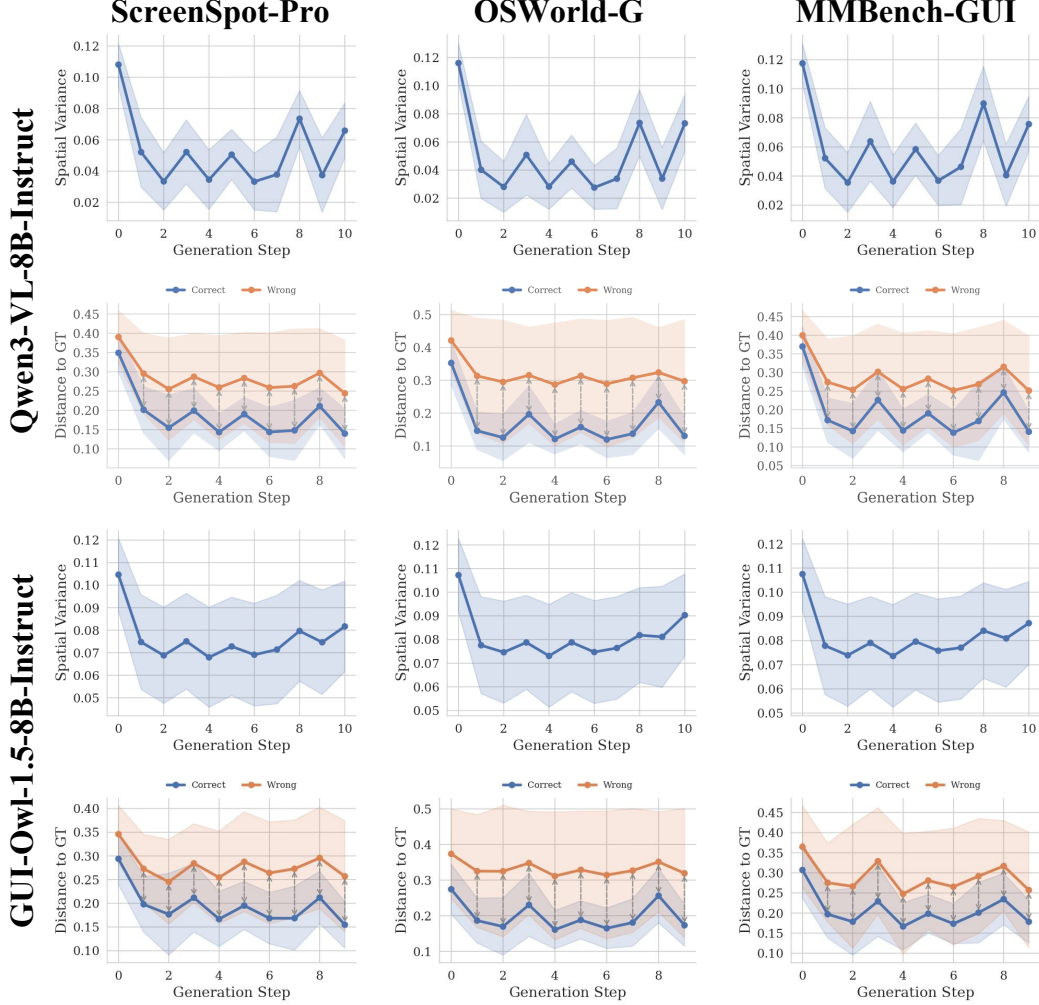


Figure 7: **Spatial variance and prefill-stage error analysis across models and benchmarks.** Rows 1–2 show results for Qwen3-VL-8B-Instruct, and Rows 3–4 for GUI-Owl-1.5-8B-Instruct. For each model, the first row shows spatial variance across generation steps, and the second row shows attention-centroid deviation from the ground truth for correct and incorrect predictions. Across all settings, attention is dispersed at prefill and rapidly contracts after the first decoding step, while errors introduced at prefill persist throughout decoding.

each model, the first row shows spatial variance across generation steps, and the second row shows attention-centroid deviation from the ground truth for correct and incorrect predictions.

Consistent patterns are observed across all models and datasets. Spatial variance is high at the prefill stage ($t = 0$), drops sharply after the first decoding step ($t = 1$), and remains low thereafter, indicating rapid concentration of attention to a single region. In parallel, incorrect predictions are already misaligned with the ground truth at $t = 0$ and remain so during decoding, whereas correct predictions stay well aligned. These results show that the two-stage inference behavior identified in Section 2.2 holds across model architectures and benchmark domains, indicating that the prefill-stage bottleneck is a general property of VLM-based GUI grounding rather than an artifact of a specific model or dataset.

E Results on OSWorld-G-Refined

This appendix presents results on OSWorld-G with the refined annotation. As shown in Table 7, Re-Prefill improves all base models under the refined annotation, with gains of +1.9 on Qwen3-

Table 7: Results on OSWorld-G (refined annotation). Methods marked with [†] are reproduced using the zoom-in strategy adopted in this work.

Model	Text Match.	Element Rec.	Layout Under.	Fine-grained Manip.	Refusal	Overall
UI-TARS-1.5-7B [27]	52.6	75.4	72.4	66.7	-	64.2
GTA1-32B [4]	63.2	83.6	84.4	70.5	-	72.2
MAI-UI-8B [6]	79.3	78.8	84.2	59.7	-	72.9
GUI-Owl-1.5-8B-Instruct [7]	73.1	69.4	71.0	74.8	42.5	69.3
<i>Training-free methods</i>						
MVP-8B [16]	82.4	76.7	79.5	61.7	-	72.7
MVP-32B [16]	81.6	76.4	77.5	62.4	-	72.0
<i>Qwen3-VL-8B-Instruct [18][†]</i>	78.9	78.2	80.2	57.7	-	71.9
+ Re-Prefill	81.2	79.1	82.2	61.7	-	73.8 _{+1.9}
<i>MAI-UI-8B [6][†]</i>	79.7	80.0	81.8	58.4	-	73.2
+ Re-Prefill	81.2	79.7	81.8	59.1	-	73.5 _{+0.3}
<i>GUI-Owl-1.5-8B-Instruct [7][†]</i>	81.6	79.4	82.6	65.1	-	74.6
+ Re-Prefill	81.6	79.7	83.4	65.8	-	75.0 _{+0.4}
<i>Qwen3-VL-32B-Instruct [18][†]</i>	82.4	85.2	87.0	62.4	-	77.3
+ Re-Prefill	83.5	84.2	85.8	65.8	-	77.5 _{+0.2}

Table 8: Comparison of computational efficiency among training-free methods. Inference time is reported as the average per-sample latency in seconds. Methods marked with [†] are reproduced under the same experimental settings in this work.

Methods	ScreenSpotPro		OSWorld-G		MMBench-GUI	
	Time (s)↓	Acc. (%)↑	Time (s)↓	Acc. (%)↑	Time (s)↓	Acc. (%)↑
<i>Qwen3-VL-8B-Instruct</i>						
MVP [†]	9.0	65.7	4.2	59.3	5.4	84.4
ZoomClick [†]	11.2	66.1	4.1	56.7	7.0	84.4
Re-Prefill	8.0	70.1	4.1	65.7	4.8	86.4
<i>Qwen3-VL-32B-Instruct</i>						
MVP [†]	11.8	73.9	6.4	65.3	8.4	87.8
ZoomClick [†]	12.4	71.5	6.2	58.2	7.6	86.4
Re-Prefill	12.2	76.8	7.5	70.1	7.8	89.5

VL-8B-Instruct, +0.3 on MAI-UI-8B, +0.4 on GUI-Owl-1.5-8B-Instruct, and +0.2 on Qwen3-VL-32B-Instruct. Compared with MVP using the same base models, Re-Prefill reaches 73.8 versus 72.7 for MVP-8B at the 8B scale and 77.5 versus 72.0 for MVP-32B at the 32B scale. These results demonstrate that the effectiveness of Re-Prefill remains consistent under refined annotations.

F Computational Efficiency Analysis

We compare the computational efficiency of Re-Prefill with existing training-free methods, including MVP [16] and ZoomClick [13], under identical base models. Experiments are conducted on Qwen3-VL-8B-Instruct and Qwen3-VL-32B-Instruct across three benchmarks, ScreenSpot-Pro, OSWorld-G, and MMBench-GUI. We report the average inference time per sample (in seconds) and the corresponding grounding accuracy. To ensure fairness, MVP and ZoomClick are reproduced under the same experimental environment and settings.

As shown in Table 8, Re-Prefill achieves the highest accuracy among training-free methods across all datasets and model scales while maintaining competitive runtime. On 8B models, Re-Prefill is slightly faster than or comparable to existing methods. On 32B models, Re-Prefill operates under a comparable computational budget to other training-free methods while achieving higher grounding performance. Overall, Re-Prefill provides a favorable efficiency–performance trade-off. It improves grounding accuracy with competitive efficiency, particularly on smaller models.

G Pseudocode of Re-Prefill

Algorithm 1 gives the complete procedure of Re-Prefill described in Section 3. Given an input sequence $\mathbf{x} = [\mathbf{S}; \mathbf{V}; \mathbf{T}]$, the algorithm performs an initial prefill, selects key visual tokens through cross-layer consistency, executes a layer-wise second prefill with hyperparameter L_c , and composes the decoding context $\mathcal{C}^*$.

Algorithm 1 Re-Prefill Inference

Require: Input sequence $\mathbf{x} = [\mathbf{S}; \mathbf{V}; \mathbf{T}]$; quantile threshold ρ ; ratio threshold γ ; continuity layers L_c
Ensure: Predicted coordinates (x, y)

```

1: Step 1: Initial prefill and attention extraction
2:  $(\mathbf{h}_q, \mathcal{C}_1, \{\mathbf{a}^{(l)}\}_{l=1}^L, [\tilde{\mathbf{S}}; \tilde{\mathbf{V}}; \tilde{\mathbf{T}}]) \leftarrow f_{\text{prefill}}(\mathbf{x})$ 

3: Step 2: Key visual token selection
4:  $\tau \leftarrow \text{quantile}(\{\mathbf{a}^{(l)}\}_{l=1}^L, \rho)$ 
5: for each visual token index  $i = 1, \dots, N_v$  do
6:    $r(i) \leftarrow \frac{1}{L} \sum_{l=1}^L \mathbb{1}[a^{(l)}(i) > \tau]$ 
7: end for
8:  $\tilde{\mathbf{V}}^* \leftarrow \{\tilde{v}_i \mid r(i) \geq \gamma\}$ 

9: Step 3: Layer-wise second prefill
10: for decoder layer  $l = 1, \dots, L$  do
11:   if  $l \leq L_c$  then
12:     prefix  $\leftarrow [\tilde{\mathbf{S}}; \tilde{\mathbf{V}}; \tilde{\mathbf{T}}]$ 
13:   else
14:     prefix  $\leftarrow [\tilde{\mathbf{V}}^*; \tilde{\mathbf{T}}]$ 
15:   end if
16:   apply decoder layer  $l$  to [prefix;  $\mathbf{S}; \mathbf{V}; \mathbf{T}$ ]
17: end for
18: Extract enriched visual representations  $\hat{\mathbf{V}}$  at the second set of visual-token positions

19: Step 4: Decoding context composition and generation
20:  $\mathcal{C}^* \leftarrow [\tilde{\mathbf{S}}; \tilde{\mathbf{V}}; \tilde{\mathbf{T}}; \hat{\mathbf{V}}]$ 
21:  $(x, y) \leftarrow f_{\text{decode}}(\mathcal{C}^*)$ 
22: return  $(x, y)$ 

```

H More Discussions

This section addresses three concerns regarding the motivation and design of Re-Prefill that are not fully unpacked in the main text.

Q1. Are spatial variance and centroid sufficient indicators of target selection?

Spatial variance and centroid characterize the attention distribution rather than directly measuring target selection. In particular, high attention does not strictly imply the correct target, as unrelated regions may also receive strong responses. This limitation is explicitly accounted for in our design. Instead of treating attention as a precise indicator, we use it as a coarse, recall-oriented signal to identify candidate regions. The key visual token selection step aims to ensure that the correct target region is included, even in the presence of noisy attention. The actual target selection refinement is then performed by the second prefill under this candidate prior. Under this design, spatial variance and centroid are not used as direct measures of target selection, but to reveal the existence and evolution of a target-selection signal during prefill.

Q2. Why does Re-Prefill not amplify first-prefill errors?

Although Re-Prefill relies on first-prefill attention to select key visual tokens, it does not amplify errors because the initial attention rarely concentrates exclusively on a wrong region. Instead, it spreads over multiple candidate UI regions, typically including the ground-truth target with non-trivial attention mass, as shown in Figure 1(a) and Appendix C. Even for incorrect predictions, the centroid

deviation remains bounded (Figure 1(c)), indicating that the correct region is not excluded. Thus, the difference between correct and incorrect cases lies in how attention is distributed among candidates rather than whether the target is covered. The second prefill re-evaluates these candidates under an explicit prior formed by key visual tokens. As long as the correct target is included, it can be promoted during re-ranking, enabling error correction rather than amplification.

Q3. Why are the performance gaps to Blind Re-Prefill and Random Token Selection in Table 6 relatively small?

The relatively small gaps to Blind Re-Prefill (0.7%) and Random Token Selection (1.0%), compared to the 4.1% gap to Embedding Addition, reflect different components of the method rather than comparable variants. The large gap to Embedding Addition isolates the contribution of the second prefill, showing that re-encoding visual tokens is the primary source of improvement. Given the second prefill, the remaining gains (0.7–1.0%) come from token selection. Blind Re-Prefill dilutes the signal by treating all visual tokens equally, while Random Selection introduces misaligned guidance. In contrast, selecting key visual tokens concentrates attention on candidate regions and aligns the guidance signal. Overall, the second prefill provides the main gain, and selection improves its effectiveness.

I Limitations

While Re-Prefill demonstrates consistent improvements across models and benchmarks, several limitations remain. As shown in the experimental results, the improvements from Re-Prefill are more pronounced in visually complex settings with high resolution and dense UI elements, where accurate target selection is the main challenge. In contrast, gains are smaller in instruction-heavy settings, where performance is more constrained by the ability to interpret complex or abstract instructions. This suggests that while Re-Prefill improves target selection during the prefill stage, it does not directly enhance instruction understanding. Extending the approach to better support complex instruction comprehension remains an important direction for future work.